TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN · BASES DE DATOS II · SEGUNDA ENTREGA

Food Store: capa de datos en PostgreSQL, MongoDB y SQLite

Grupo 10 (Lobos Solitarios) · Mariano Chirino, Andrés Fabre, Facundo Quiroga

Entrega: jueves 12 de noviembre de 2026, 23:59

Repositorio: https://github.com/marianoemir/TPI-BASE-DE-DATOS-2

Etiqueta final: `tp-final`

Contenido. 1. Introducción y alcance · 2. Los 14 objetivos y dónde está la evidencia · 3. Decisiones de diseño por unidad (U4, U5, U6, U7, U8 y code review) · 4. Uso de IA · 5. Limitaciones y trabajo futuro · 6. Bases de datos, cómo reproducir y notas

1. Introducción y alcance

Food Store es una base de datos de un comercio de alimentos que se trabajó a lo largo de la cursada en cinco unidades: normalización (U4), seguridad (U5), mantenimiento y recuperación (U6), modelado documental (U7) y base local para una tablet (U8). Esta segunda entrega no agrega un sistema nuevo: **cierra y justifica** lo hecho, y prueba de forma verificable el cumplimiento de los 14 objetivos.

Motores: PostgreSQL 16+ (PL/pgSQL), MongoDB y SQLite. Equipo: Mariano Chirino, Andrés Fabre y Facundo Quiroga. El repositorio integrador está organizado por unidad; el `README.md` de la raíz trae la tabla de los 14 objetivos con la ruta de cada evidencia. Link: https://github.com/marianoemir/TPI-BASE-DE-DATOS-2. Etiqueta: `tp-final`.

Criterio de lectura: **el informe justifica y el repositorio prueba**. Cada afirmación apunta a un archivo o a una salida guardada. Lo que no se pudo demostrar se dice como límite, no se omite.

Nota sobre el PDF de la U7. `unidad-7/informe_u7.pdf` es la entrega original de la unidad (01/11/2026) y enlaza al repositorio `marianoemir/TP-Unidad7-BD2`. No incluye la medición de 5.000 pedidos, que está en `unidad-7/evidencias/comparacion_medida.md` y en la sección de la U7 de este informe. No se editó: se revisó página por página (texto, metadatos y capturas) y no contiene rutas locales ni nombres de equipo; solo los nombres de los integrantes y datos ficticios de `data.sql`.

Cómo está organizado el repositorio

Carpeta	Contenido
`heredado/`	Base de partida (`base_heredada/`) y material de la U7 (`u7_mongodb/`)
`unidad-4/`	Normalización (FNBC), desnormalización controlada, migración
`unidad-5/`	Roles y permisos, auditoría de acceso, simulacro de incidente, auditoría de `roles.sql` asistida por IA
`unidad-6/`	Respaldo, restauración, mantenimiento, simulacro de desastre
`unidad-7/`	Modelo documental (MongoDB) y comparación con PostgreSQL
`unidad-8/`	Base SQLite de la tablet, sincronización simulada, semáforo de conflictos
`docs/`	Informe, bitácora de IA unificada, auditoría de código

2. Los 14 objetivos y dónde está la evidencia

#	Objetivo	Estado	Evidencia	Límite conocido
1	Refactor a BCNF validado contra dependencias funcionales	Cubierto	`unidad-4/tp_fnbc_control_lote.sql`, `unidad-4/specs/spec_parte_1_fnbc_control_lote.md`	La dependencia `{L,D}→R` no se preserva (documentado)
2	Migración sin pérdida, transaccional, con plan de reversión	Cubierto con límites (en `main`, PR #18)	`unidad-4/migracion/` (`setup_previo.sql`, `up.sql`, `down.sql`, `verificar_migracion.sql`, `pruebas_falla.sql`), `unidad-4/capturas/migracion_*`, `docs/informe/u4.md`	Spec posterior al código; sin pasada previa con `ROLLBACK`; no preserva `{L,D} -> R`; contraer pendiente; dos fuentes de verdad; sin permisos para los roles de la U5; `down` repetido sin captura propia; escala de 3 filas
3	Desnormalización controlada documentada	Cubierto	`unidad-4/tp_desnormalizacion_top_categorias.sql`, `unidad-4/capturas/explain_*.txt`	La consulta base tarda 6,5 ms: difícil de justificar como «lenta»
4	Roles y permisos de mínimo privilegio	Cubierto	`unidad-5/sql/roles.sql`, `unidad-5/docs/kiro_roles_spec/`, `unidad-5/docs/verificacion_permisos.md` (T01–T23: 23/23)	No cubre los objetos de la U4 (`control_lote`, `responsable_deposito`, `mv_ventas_categoria_dia`)
5	Auditoría y simulacro de acceso indebido, con contención	Cubierto	`unidad-5/docs/informe_auditoria.md`, `unidad-5/sql/simulacro_incidente.*`, `unidad-5/docs/informe_incidente.md`	Evidencia repartida en tres servidores y fechas
6	Auditoría de permisos asistida por IA, con decisión humana	Cubierto	`unidad-5/docs/auditoria_roles_opencode.md`, `unidad-5/docs/registro_opencode_auditoria_roles_corrida2.txt`, `unidad-5/evidencias/obj6_*`	Repetición del 07/11: la sesión original no se conserva
7	Plan de mantenimiento completo	Cubierto con límites	`unidad-6/` (`backup_foodstore.ps1`, `restore_test.ps1`, `programar_tareas.ps1`, `validar_integridad.sql`, `SPEC_U6_ampliacion.md`), `unidad-6/evidencias/obj7_*_20261107.txt`, `docs/informe/u6.md`	Tareas registradas y ejecutadas por el Programador solo por disparo manual (ninguna por horario); `Interactive` sin `-StartWhenAvailable`; el mantenimiento sobre `foodstore` solo observa (las acciones se ensayan en la base descartable); respaldo y mantenimiento corren como `postgres`; `pg_hba` en `trust`; Pester solo busca texto
8	Simulacro de desastre contra reloj	Cubierto	`unidad-6/restore_test.ps1`, `unidad-6/evidencias/restore_log_ok.txt`, `informe_u6.pdf`	RTO 6,19 s sobre ~10,6 MB, mismo equipo
9	Modelado documental con consultas verificadas	Cubierto	`unidad-7/modelo/esquema_documental.md`, `unidad-7/consultas_mongodb.js`	Escala de laboratorio (7 documentos)
10	Comparación documental vs relacional	Cubierto con límites	`unidad-7/evidencias/comparacion_medida.md`, `unidad-7/evidencias/obj10_*_20261107.txt`, `unidad-7/SPEC_U7_m`	Una sola máquina y una muestra de 5.000 pedidos; el producto más frecuente aparece en un solo pedido; en la repetición no se capturó el plan de PostgreSQL; tiempos no comparables 1 a 1 con `EX`

			`edicion.md`, `docs/informe/u7.md`	`PLAIN ANALYZE`
11	SQLite especificado en Kiro y generado con OpenCode	Cubierto con límites	`unidad-8/.kiro/specs/foodstore-tablet-sync/`, `unidad-8/movil_opencode/`, `unidad-8/movil_opencode_v2/`, `unidad-8/docs/comparacion_generado_vs_original.md`	Ninguna corrida de OpenCode produjo una base funcional sin correcciones
12	Sincronización simulada con conflictos	Cubierto	`unidad-8/movil_opencode/semaforo.sql`, casos C1–C8 y F2, `unidad-8/evidencias/01` a `29`	El semáforo evalúa cada borrador por separado; solo el servidor resuelve la competencia por stock (C8)
13	Code review final asistido por IA	Cubierto con límites	`docs/informe/codereview.md`, `docs/auditoria_codigo/` (`hallazgos.md`, `tanda_*.md`, `evidencias/`)	196 hallazgos; 42 decididos (35 altas o críticas; 14 con ejecución propia, el resto por lectura); 154 «No evaluado»; sin correcciones de código; sin revisión cruzada independiente
14	Capa de datos completa en Git, lista para entrega	Cubierto con límites	Este README, `docs/bitacora_ia.md`, `docs/informe/`	Sin revisión cruzada en varios PR; 154 de 196 hallazgos del code review sin evaluar; evidencias con rutas locales sin editar (ver sección 5)

3. Decisiones de diseño por unidad

U4 — FNBC, migración y desnormalización (objetivos 1 a 3)

Alcance y autoría

La U4 tiene tres partes, que corresponden a los objetivos 1 a 3:

Objetivo	Qué es	Archivos principales
1	Refactor a FNBC de `control_lote_almacen`	`unidad-4/tp_fnbc_control_lote.sql`, `specs/spec_parte_1_fnbc_control_lote.md`
2	Migración transaccional y reversible	`unidad-4/migracion/`
3	Desnormalización controlada del reporte «top 5 categorías del día»	`unidad-4/tp_desnormalizacion_top_categorias.sql`, `specs/parte2_desnormalizacion_top_categorias.md`

Según las DUIA (`unidad-4/duia/`): el diagnóstico teórico de la spec de la Parte 1 lo generó Andrés; la versión final del script de la Parte 1 y la Parte 2 las consolidó Mariano; la migración `up`/`down` (objetivo 2) la hizo Facundo con Claude.

Objetivo 1 — FNBC sobre `control_lote_almacen`

Qué hicimos

Analizamos `control_lote_almacen(lote_id, deposito_id, responsable_control_id)` con dos dependencias funcionales, tomadas del enunciado:

- F1: `{L, D} → R` (un lote en un depósito tiene un único responsable).
- F2: `R → D` (cada responsable pertenece a un único depósito).

Las claves candidatas son `{L, D}` y `{L, R}`, y los tres atributos son primos. La relación **no está en FNBC**, porque en F2 el determinante `R` no es superclave (`{R}+ = {R, D}`). Sí cumple 3FN, porque en F2 el atributo determinado (`D`) es primo y 3FN admite esa excepción.

Mostramos las tres anomalías sobre la instancia del enunciado (actualización del responsable 801, inserción del 803, borrado del 802) y descompusimos sin pérdida en:

- `responsable_deposito(responsable_control_id PK, deposito_id)`
- `control_lote(lote_id, responsable_control_id, PK compuesta)`
- una vista de compatibilidad `v_control_lote_almacen` con `NATURAL JOIN`, que reconstruye la relación original.

Por qué así

- **FNBC y no solo 3FN.** La relación ya cumple 3FN, pero F2 sigue permitiendo las tres anomalías. El costo es que la descomposición **no preserva F1**, y lo dejamos documentado como límite.
- **Vista aparte y tabla original intacta.** La spec original dejaba abierta una alternativa (renombrar la tabla original y crear una vista con su nombre). El grupo decidió la vista aparte, sin renombrar la tabla original, y así quedó en la spec.

Cómo lo probamos

- Conteo de filas (3 y 3) y los **dos `EXCEPT`**, en los dos sentidos (Paso 5 del protocolo), con 0 filas cada uno.
- Anomalía de actualización reproducida dentro de `BEGIN … ROLLBACK`: el motor acepta el cambio parcial y el responsable 801 queda en dos depósitos.
- Limitación de la descomposición reproducida también con `ROLLBACK`: el lote 501 queda con dos responsables en el depósito 30.

Resultado y dónde está la evidencia

Script completo ejecutado sin errores. Capturas en `unidad-4/capturas/`: `p1_00_script_completo_ok.png`, `p1_captura1_original.png`, `p1_captura3_anomalia_actualizacion.png`, `p1_captura4a_responsable_deposito.png`, `p1_captura4b_control_lote.png`, `p1_captura5_vista.png`, `p1_captura6_except_original_menos_vista.png`, `p1_captura7_except_vista_menos_original.png`, `p1_captura8_conteo.png`, `p1_captura9_limitacion_no_preserva_F1.png`, `p1_down_1_antes.png`, `p1_down_2_despues.png` y `p1_down_3_up_de_nuevo.png`.

Límite conocido

- **No se preserva `{L, D} → R`.** El esquema nuevo acepta dos responsables para el mismo lote y depósito. Con más tiempo agregaríamos un trigger o una consulta de auditoría que la haga cumplir.
- **`R → D` es una regla de negocio asumida.** Que la consulta de verificación devuelva 0 filas solo confirma que se cumple en los datos de hoy; no prueba que sea una regla.
- **Escala mínima:** la instancia del enunciado tiene 3 filas.

Objetivo 2 — Migración transaccional y reversible

Qué hicimos

Convertimos la descomposición a FNBC de `control_lote_almacen` (que viola FNBC por `ResponsableControlID → DepositoID`) en una migración con dos archivos, como exige el Paso 6 de `protocolo_seguridad.md`, cada uno dentro de `BEGIN…COMMIT`. Están en `unidad-4/migracion/`:

Archivo	Para qué sirve
`setup_previo.sql`	Deja la base en el estado previo: `lote`, `deposito`, responsables 801 a 803 y `control_lote_almacen` con la instancia del enunciado. Se puede re-ejecutar y solo se usa en copias de trabajo.
`up.sql`	Expandir y migrar: crea `responsable_deposito`, `control_lote` y la vista `v_control_lote_almacen`, copia los datos y verifica. No toca la tabla original.

`down.sql`	Elimina la vista y las dos tablas nuevas. `control_lote_almacen`, `lote`, `deposito` y `usuario` quedan intactos.
`verificar_migracion.sql`	Consultas de solo lectura (V1 a V5) para comprobar el estado después de `up` y de `down`.

Por qué así

- **Dos archivos y no uno.** La versión anterior era un solo script, con el `down` comentado y el `up` sin transacción, lo que no cumplía nuestro propio protocolo. Separar permite probar `up` y `down` por su cuenta y que cada uno sea atómico.
- **Verificación dentro del `up`.** Los dos `EXCEPT` (Paso 5) y el conteo corren al final del `up`, antes del `COMMIT`. Si algo no coincide, se lanza una excepción y no se confirma nada. Descartamos dejar la verificación solo como consultas manuales posteriores, porque obliga a recordar ejecutarlas y deja la migración confirmada aunque esté mal.
- **Precondición sobre `R → D`.** Antes de crear nada, el `up` comprueba que ningún responsable figure en más de un depósito. Si pasa, falla con un mensaje claro en lugar de un error de clave duplicada, y se resuelve con el negocio, sin forzar el script (Paso 6, etapa 2).
- **Sin `IF NOT EXISTS` en el `up`.** Si las tablas ya existen, el `up` falla en vez de quedar a medias. El `down`, en cambio, usa `IF EXISTS` y se puede repetir sin riesgo.
- **No contraemos.** La tabla original se conserva, como indica el Paso 6 (la etapa de contraer va en una migración posterior) y para poder comparar contra la vista.

Cómo lo probamos

1. En `copia_u4` (pgAdmin 4) cargamos `heredado/base_heredada/` (`schema.sql`, `objects.sql`, `data.sql`; conteos esperados: 6 pedidos, 13 detalles, 16 productos).
2. Respaldo previo (Paso 3): `copia_u4_20261108_1638.sql`, sacado el 08/11/2026 (guardado fuera del repo, según el Paso 3), después de `setup_previo.sql` y antes de `up.sql`.
3. Orden de la corrida (08/11/2026): `setup_previo.sql`, respaldo, `up.sql`, V1 a V4, `down.sql`, V1 y V5. La primera pasada se hizo sin respaldo previo; se sacó el respaldo y se repitió el ciclo completo (detalle en la DUIA de Facundo).
4. Resultado de `up.sql` en pgAdmin: `Verificación OK: perdidas=0, espurias=0, filas antes=3 / despues=3`, con `COMMIT`. Tras `down.sql`, V1 devolvió 0 filas y V5 las 3 filas originales.
5. Los dos caminos de falla se probaron primero en un PostgreSQL 16 local, en el chat de Claude, sin captura. El 08/11/2026 Mariano Chirino los reprodujo en pgAdmin (`copia_u4`, PostgreSQL 17) y guardó la evidencia: `up.sql` ejecutado dos veces (falla con «la relación ya existe» y no deja nada a medias) y `up.sql` con una fila que pone a un responsable en dos depósitos (se frena en la precondición y no crea ningún objeto). Los pasos están en `unidad-4/migracion/pruebas_falla.sql`. El `down.sql` repetido sigue sin captura propia. Versiones: las capturas del ciclo `up/down` (`migracion_up.png`, `migracion_down.png`) muestran PostgreSQL 18 en la barra de conexión de pgAdmin (las subió la cuenta `Buhonero009` en la rama de Facundo, commit `5a2472f`); las de los caminos de falla muestran PostgreSQL 17 (corridas de Mariano). Las pruebas no se repitieron en una misma versión.

Resultado y dónde está la evidencia

Las capturas están en `unidad-4/capturas/`.

Comprobación	Esperado	Evidencia
V1 tras `up`: objetos creados	3 filas	`migracion_v1_tras_up.png`
V2 y V3: `EXCEPT` en los dos sentidos	0 filas cada uno	`migracion_v2_tras_up.png` y `migracion_v3_tras_up.png`

V4: conteos original / vista / `responsable_deposito` / `control_lote`	3 / 3 / 2 / 3	`migracion_v4_tras_up.png`
V1 tras `down`	0 filas	`migracion_v1_tras_down.png`
V5 tras `down`: tabla original intacta	3 filas	`migracion_v5_tras_down.png`
`up.sql` completo	`COMMIT` y aviso `Verificación OK: perdidas=0, espurias=0, filas antes=3 / despues=3`	`migracion_up.png`
`down.sql` completo	`COMMIT` sin errores	`migracion_down.png`
`up.sql` repetido falla sin dejar nada a medias	Error «la relación responsable_deposito ya existe» (42P07); V1 después del rollback devuelve las mismas 3 filas	`migracion_falla_up_repetido.png` y `migracion_falla_up_repetido_v1.png`
`up.sql` con datos que violan `R → D` se frena en la precondición	Error «Precondición violada…» (P0001); V1 después del rollback devuelve 0 filas	`migracion_falla_precondicion.png` y `migracion_falla_precondicion_v1.png`

Archivos: `unidad-4/migracion/up.sql`, `down.sql`, `setup_previo.sql`, `verificar_migracion.sql`.

Límite conocido

- **No se preserva `{LoteID, DepositoID} → ResponsableControlID`.** La descomposición es sin pérdida pero no preserva F1: el esquema nuevo acepta dos responsables para el mismo lote y depósito. Con más tiempo agregaríamos un trigger o una consulta de auditoría que la haga cumplir.
- **El `down` descarta lo escrito después del `up`** en las dos tablas nuevas. Como la original se conserva, no se pierde el estado previo, pero sí lo cargado solo en las tablas nuevas.
- **Escala mínima.** Probamos con las 3 filas del enunciado, no con volumen.
- **Contraer queda pendiente.** La tabla original sigue existiendo.
- **Dos fuentes de verdad mientras coexistan.** `v_control_lote_almacen` lee solo `control_lote` y `responsable_deposito`: una escritura posterior al `up` sobre `control_lote_almacen` no aparece en la vista, y una escritura en las tablas nuevas no llega a la original.
- **Permisos sin cubrir.** `up.sql` no otorga privilegios sobre las tablas ni la vista nuevas, y los roles de la U5 (`unidad-5/sql/roles.sql`) no las incluyen, porque la ampliación de roles a los objetos de la U4 (objetivo 4) no se hizo.
- **Sin pasada previa con `ROLLBACK` (Paso 2).** El protocolo pide ejecutar primero cada script terminando en `ROLLBACK` y repetirlo con `COMMIT` solo después de inspeccionar el efecto. Corrimos `up.sql` directamente con `COMMIT`. Lo compensamos con tres controles: respaldo previo con `pg_dump` (Paso 3), verificación dentro de la transacción (si algo no coincide se lanza una excepción y no se confirma nada) y `down.sql` probado después del `up`. Con más tiempo haríamos la pasada con `ROLLBACK` antes de la definitiva.
- **Orden spec/código.** La spec de Kiro se actualizó con el plan de reversión en el commit `b55a4cb` (2026-11-08), después de escribir y ejecutar los archivos de migración. El historial de Git no lo refleja, porque ese commit se hizo antes de commitear `migracion/`. El orden real fue: archivos de migración y corrida en `copia_u4`, luego la spec y recién después los commits. Lo declaramos acá igual que en el objetivo 11.

Objetivo 3 — Desnormalización controlada: top 5 categorías del día

Qué hicimos

El panel de administración consulta, «muchas veces por minuto», las 5 categorías con más ventas del día.

La consulta original une 4 tablas (`detalle_pedido`, `producto`, `categoria`, `pedido`). Agregamos la **vista materializada** **`mv_ventas_categoria_dia`** (agregada por categoría y fecha) con un índice único `(categoria_id, fecha)`, que permite refrescarla con `REFRESH MATERIALIZED VIEW CONCURRENTLY` sin bloquear lecturas. Las tablas fuente no se modifican.

Por qué así

- **Vista materializada y no columna + trigger.** El reporte es un agregado (`SUM`, `GROUP BY`, `ORDER BY`, `LIMIT`), no un valor por fila: un trigger tendría que recalcularlo en cada escritura de `detalle_pedido`. El panel lee mucho y tolera una ventana chica de desactualización, a diferencia de un dato crítico como el stock. Además ya existía un precedente en el proyecto (`mv_facturacion_categoria_mes`) con el mismo mecanismo.
- **Los cuatro requisitos del Paso 7:**

Requisito	Cómo se cumple
Motivo medido	`EXPLAIN (ANALYZE, BUFFERS)` antes: 6,546 ms y 4.464 buffers
Un único dueño del dato	Solo `REFRESH MATERIALIZED VIEW CONCURRENTLY` escribe la vista; ningún trigger
Consulta de conciliación	Auditoría con `FULL OUTER JOIN … IS DISTINCT FROM`, que devuelve 0 filas
Documentado y reversible	`DROP MATERIALIZED VIEW`; la fuente de verdad queda intacta

Cómo lo probamos

- **Escenario.** La carga masiva solo poblaba fechas históricas, así que `CURRENT_DATE` casi no tenía volumen. En `copia_trabajo` (descartable) se reasignaron a `CURRENT_DATE` los 400 pedidos del 2025-10-09 y se corrió `ANALYZE`. Quedaron 406 pedidos del día (405 vigentes, 811 detalles, 5 categorías), y la consulta del enunciado se midió tal cual.
- **Medición.** Tercera corrida, con caché caliente.

Momento	Tiempo	Buffers	Nodo dominante
Antes	6,546 ms	4.464	`Nested Loop` (`loops=405` y `loops=811`)
Después	0,106 ms	13	`Bitmap Index Scan` sobre el índice único

Mejora de unas 62 veces en tiempo y 340 veces en bloques leídos.

- **Primera medición descartada.** Se había sustituido la fecha por `'2025-01-01'` y dio 1.301,956 ms, pero fue en caché fría y con solo 2 categorías, así que no era comparable con la del «después».
- **Auditoría.** 0 filas. Probamos que detecta un desfasaje: dentro de `BEGIN … ROLLBACK`, un `UPDATE` sobre `detalle_pedido` hizo que devolviera 1 fila (diferencia de 52.500,00), y tras el `ROLLBACK` volvió a 0 filas. La primera versión de la auditoría usaba `JOIN` interno y daba un falso negativo; se reemplazó por `FULL OUTER JOIN`.

Resultado y dónde está la evidencia

`unidad-4/tp_desnormalizacion_top_categorias.sql`, `specs/parte2_desnormalizacion_top_categorias.md`, `capturas/explain_antes.txt`, `capturas/explain_despues.txt` y `capturas/p2_auditoria_0_filas.png`.

Límite conocido

- **El «antes» es difícil de defender como «lento».** 6,5 ms no es una consulta lenta en términos absolutos. El argumento que sí se sostiene es relativo: 4.464 buffers para devolver 5 filas, un costo que crece con los pedidos del día, y una frecuencia de «muchas veces por minuto». **No probamos un índice cubriente sobre la consulta original**, que podría acercar su tiempo al de la vista; con más tiempo lo mediríamos y compararíamos.
- **El `REFRESH` recalcula todo el historial:** 6,8 s sobre ~200.000 pedidos. Es el costo de mantenimiento

de esta estrategia. No hay `pg_cron` en el entorno, así que el refresco programado queda como decisión pendiente.
- **Ventana de inconsistencia:** la vista muestra los datos del último `REFRESH`.
- **Medición solo reproducible el mismo día** del `UPDATE` a `CURRENT_DATE`.
- **Respaldo previo (Paso 3).** En la ejecución original no se sacó respaldo previo y no se repitió; el comentario del script se corrigió para decirlo.

Uso de IA

- **Primer borrador descartado.** El primer `up.sql` era una plantilla genérica (creaba `detalle_operacion` con una FK a una tabla `operacion` que no existe en Food Store) y falló al ejecutarlo. No lo usamos. Esa plantilla se generó, según la DUIA de Facundo, con Gemini (el modelo exacto quedó sin confirmar).
- **Versión final.** Los cuatro archivos los generó Claude (chat) a partir de `tp_fnbc_control_lote.sql` (secciones 1, 4, 5 y 8). Lo que se agregó respecto del script original: la precondición, la verificación dentro de la transacción y `verificar_migracion.sql`.
- **Control humano.** Facundo leyó los archivos línea por línea y los ejecutó en `copia_u4`; la salida está guardada como evidencia.
- **Redacción del informe.** Este informe y la sección 9 de la spec se redactaron con ayuda de Claude (chat) a partir de los archivos del repositorio, y Facundo los revisó y completó. Las secciones de los objetivos 1 y 3 describen trabajo de otros integrantes (ver «Alcance y autoría»). El detalle está en `unidad-4/duia/duia_facundo.md`.

Unidad 5: roles, permisos y auditoría (objetivos 4, 5 y 6)

Autor: Mariano Chirino. Base de pruebas: `copia_u5`, PostgreSQL 17.10. Rama: `m/obj-06`.

1. Roles y mínimo privilegio

Qué hicimos. `unidad-5/sql/roles.sql` crea cinco roles de grupo sin login (`rol_app_lectura`, `rol_app_escritura`, `rol_app_auth`, `rol_soporte`, `rol_reportes`) y dos roles con login: `admin_datos`, dueño de los objetos y único con `CREATE` sobre `public`, y `app_web`, que no tiene privilegios propios y solo recibe tres membresías (lectura, escritura y autenticación). Los privilegios se otorgan por rol de grupo y por objeto, se revoca `EXECUTE` a `PUBLIC` en las rutinas del proyecto, y los privilegios por omisión de `admin_datos` impiden que sus objetos futuros nazcan abiertos.

Por qué así. Los permisos van en los grupos y el login solo hereda, de modo que cambiar lo que puede hacer la aplicación no obliga a tocar al usuario. La alternativa descartada fue darle a `rol_app_escritura` permisos por columna sobre `usuario`, `producto`, `pedido` y `detalle_pedido`, y a `rol_app_auth` un `SELECT` sobre `usuario.contrasena`: amplía la superficie sin necesidad y deja los hashes al alcance del backend. En su lugar, `sp_crear_pedido` y `fn_autenticar` pasan a `SECURITY DEFINER` con `search_path` fijo y con `admin_datos` como dueño (no `postgres`, para que la rutina no corra con privilegios de superusuario), de modo que `app_web` ejecuta rutinas y no toca tablas (decisiones D1, D2 y D6 de `design.md`). Con el mismo criterio se descartaron crear `admin_datos` con `CREATEDB CREATEROLE`, como en el apunte de la clase (D3), y darle a soporte `SELECT` sobre `v_usuario_seguro`, que expone `celular` sin que el equipo lo hubiera decidido (D5). La especificación se escribió primero en Kiro (`docs/kiro_roles_spec/`) y el script se ajusta a ella.

Cómo lo probamos. `unidad-5/sql/verificacion_roles.sql` ejecuta 23 pruebas (T01 a T23) como cada rol dentro de transacciones reversibles: qué puede leer, qué se le deniega, que `app_web` crea pedidos y se autentica, que `rol_soporte` no puede crear pedidos, y que `app_web` no tiene membresías extra.

Resultado y evidencia. 23/23 `OK` con la versión final de `roles.sql` (`unidad-5/evidencias/t01_t23_roles_final.txt`). El detalle de cada prueba está en `unidad-5/docs/verificacion_permisos.md`.

Límites conocidos.

- La copia de pruebas usa `trust` en `pg_hba.conf`, así que T01 a T23 verifican permisos y no la autenticación por contraseña. En producción haría falta `scram-sha-256`.
- Los roles se crean con la contraseña `'cambiar_en_produccion'`, que es pública en el repositorio, hasta que alguien la cambie.
- Los roles **no se ampliaron a los objetos de la unidad 4** (`control_lote`, `responsable_deposito`, `mv_ventas_categoria_dia`): `roles.sql` se escribió y se probó sobre la base heredada, anterior a esa unidad, y no se verificó cómo quedan esos objetos frente a los siete roles.
- `app_web` puede crear un pedido para cualquier `p_usuario_id`, porque el procedimiento no valida que corresponda al usuario conectado. La spec lo registra como riesgo aceptado en esta unidad.

2. Auditoría de `roles.sql` con IA

Qué hicimos. Se pidió a OpenCode (modo Plan, solo lectura, modelo Nemotron 3.5 Lightning) que auditara `roles.sql` contra `design.md` desde el principio de mínimo privilegio. Devolvió **5 hallazgos**. Cada uno se reprodujo en `copia_u5` antes de decidir. Además, una revisión hecha con Claude aportó un sexto hallazgo (H4b). El documento completo, con prompt, registro crudo y decisión de cada hallazgo, es `unidad-5/docs/auditoria_roles_opencode.md`.

Hallazgo	Origen	Decisión	Por qué
H1 (alto): `CREATE` sobre `public`	OpenCode	Aceptado y corregido	No se veía en la copia limpia, pero un `GRANT CREATE` previo sobrevivía a `roles.sql`
H2 (medio): contraseña no restablecida	OpenCode	Descartado	El script no pisa claves existentes; el hash no cambia al re-ejecutar
H3 (medio): membresías entre grupos	OpenCode	Aceptado y corregido	`rol_soporte` heredaba `EXECUTE` sobre `sp_crear_pedido`
H4 (medio): `EXECUTE` indirecto de `PUBLIC`	OpenCode	Descartado	0 filas para `PUBLIC` en las tres rutinas del proyecto
H5 (bajo): `ALTER DEFAULT PRIVILEGES`	OpenCode	Descartado	Una función de `admin_datos` nace cerrada; el control de `postgres` nace abierto, como se espera
H4b: rutinas anteriores abiertas	Claude	Aceptado y corregido	`app_web` ejecutaba cuatro rutinas por herencia de `PUBLIC`

Por qué así. La auditoría de la IA se trató como una lista de hipótesis que hay que reproducir, no como un veredicto. La alternativa descartada era aplicar sus correcciones tal cual: la de H2 habría sobrescrito en cada ejecución la clave real de un servidor con el marcador público, y las de H4 y H5 apoyaban su razonamiento en afirmaciones que la ejecución contradijo (por ejemplo, que desde PostgreSQL 15 `EXECUTE` para `PUBLIC` ya no es el valor por omisión).

Cómo lo probamos. Cada hallazgo tiene un script y su salida en `unidad-5/evidencias/`. Los dos aceptados de OpenCode (H1 y H3) dependían de residuos en una base sucia, así que se reprodujeron sembrando el residuo, y después de corregir se repitió la misma siembra con scripts reproducibles (`unidad-5/sql/prueba_h1_correccion.ps1` y `prueba_h3_correccion.ps1`). H4b se probó antes y después del `REVOKE`, con T01 a T23 sin diferencias y con la creación de un pedido como `app_web`.

Resultado y evidencia. Las tres correcciones (H1, H3, H4b) están en `roles.sql` dentro de su transacción, con T01 a T23 en 23/23 después de cada una. Los archivos de evidencia están citados por hallazgo en `auditoria_roles_opencode.md`.

Límites conocidos.

- `sp_crear_pedido` es `SECURITY DEFINER`, así que la prueba de H4b no ejercita una ruta en la que `app_web` dispare las funciones con sus propios permisos.
- La corrección de H3 quita las membresías concedidas por `postgres`; las concedidas por otro otorgante no se probaron.
- La prueba previa a la corrección de H1 se hizo en consola y su salida no se guardó; está declarada en el documento.
- La auditoría original, anterior a esta repetición, no conserva su respuesta cruda. La repetición llegó en tres partes por dos interrupciones, con mensajes redactados con asistencia de Claude.

3. Auditoría de acceso e incidente (objetivo 5)

3.1 Registro de actividad del motor

Qué hicimos. Se activó el registro nativo de PostgreSQL con tres parámetros: `log_connections` y `log_disconnections` pasaron de `off` a `on`, y `log_statement` de `none` a `all` (`ALTER SYSTEM` más `pg_reload_conf()`). `logging_collector` ya estaba en `on`. Los valores originales se verificaron con `pg_settings` antes de cambiarlos, y el informe incluye la reversión (`ALTER SYSTEM RESET`).

Por qué así. La consigna pide como mínimo un nivel que registre modificaciones (`mod`). Se eligió `all` porque con `mod` no quedan registrados los `SELECT`, y justamente los intentos de `fn_autenticar` y la lectura masiva de `usuario` son `SELECT`.

Cómo lo probamos. `unidad-5/sql/auditoria_test.ps1` corre `auditoria_test.sql` con `psql -f`, para que cada sentencia llegue por separado y quede en su propia línea del log: una lectura, una inserción, una actualización y una llamada a `fn_autenticar` con un mail inexistente, todo dentro de una transacción con `ROLLBACK`, y después un intento de conexión como `app_web` con contraseña incorrecta. Un primer intento desde pgAdmin se descartó: todo el script llegaba como una sola sentencia y no había ningún rechazo de conexión.

Resultado y evidencia. El fragmento `unidad-5/docs/log_auditoria_prueba.txt` (ejecución del 14/10/2026, 20:55:26) permite responder quién (login y base), qué (texto completo de cada sentencia, incluidos `BEGIN` y `ROLLBACK`), cuándo (fecha y hora con segundos, más la duración de la sesión) y desde dónde (dirección y puerto, método `scram-sha-256` y regla de `pg_hba.conf`). Hay dos clases de login fallido y el log las distingue: `fn_autenticar` con credenciales incorrectas solo deja la sentencia, sin indicar si devolvió filas; una conexión con contraseña incorrecta deja un `FATAL` con el usuario y la regla aplicada. El informe completo es `unidad-5/docs/informe_auditoria.md`.

Límites conocidos.

- No hay detalle fila por fila: eso lo daría la extensión `pgaudit`.
- Las líneas `sentencia:` no traen usuario, base ni sesión, porque el prefijo del log tiene solo fecha y hora; con sesiones simultáneas la atribución se vuelve ambigua. Un `log_line_prefix` más completo lo resolvería.
- El log registra la intención y no el resultado: el `INSERT` y el `UPDATE` aparecen aunque terminaron en `ROLLBACK`.
- No identifica al usuario de la aplicación, solo al login del motor.
- Con `log_statement = 'all'` el archivo contiene mails y claves en texto plano, así que es un activo sensible: sus permisos los define el sistema operativo, no el motor, y no se versiona ni se comparte sin anonimizarlo.

3.2 Simulacro de acceso indebido

Qué hicimos. `unidad-5/sql/simulacro_incidente.sql` (orquestado por `simulacro_incidente.ps1`) ejecuta cada paso conectado como el login que corresponde:

Paso	Login	Qué se ejecutó	Resultado real
0	`postgres`	Preparación: usuario de prueba con hash real y login `soporte_sim` (miembro de `rol_soporte`)	Sin errores
1	`app_web`	5 llamadas a `fn_autenticar` con clave incorrecta	5 resultados con 0 filas
2	`app_web`	`fn_autenticar` con la clave correcta	1 fila
3	`soporte_sim`	`SELECT` de `usuario` sin `LIMIT`, solo con las columnas permitidas	20.007 filas
4	`soporte_sim`	`GRANT admin_datos`, `SET ROLE admin_datos`, `CREATE ROLE`	Los tres con permiso denegado

El log se capturó el 12/10/2026 (08:21:56 a 08:22:04) en el servidor de Andrés, con `log_statement = 'all'`; la constancia de resultados es de la ejecución de Mariano (`unidad-5/docs/constancia_escalamiento.txt`).

Por qué así. El log debía permitir reconstruir lo ocurrido sin acceso a quien lo ejecutó, así que se le pidió a OpenCode una reconstrucción y una hipótesis de contención **solo a partir del log anonimizado**, y después se la contrastó con lo que realmente se ejecutó. La anonimización (`unidad-5/docs/log_simulacro_anonimizado.txt`) reemplaza mails, claves y host por valores sintéticos y conserva roles del motor, horas y texto de las sentencias, que es lo necesario para auditar. El mapa de valores originales vive aparte, en `criterio_anonimizacion_log.md`, y no se le entrega a la IA.

Cómo lo probamos. Hubo dos corridas de OpenCode. La primera (`respuesta_opencode_simulacro_v1.md`) se descartó como evidencia: el archivo incluía un encabezado con el mapa de valores originales y la respuesta usó información que solo estaba allí. La segunda (`respuesta_opencode_simulacro.md`) usó el log sin encabezado, 70 líneas, en una carpeta vacía, y es la que se contrasta.

Resultado y evidencia. Contraste completo en `unidad-5/docs/informe_incidente.md`:

- **Acertó:** contó las 5 sesiones en unos 8 segundos, interpretó bien las 5 llamadas a `fn_autenticar`, vio que la lectura masiva pidió solo columnas permitidas, reconstruyó los tres intentos de escalamiento y su causa (falta de la opción `ADMIN` y del atributo `CREATEROLE`), y reconoció los límites del log.
- **Omitió** el volumen de filas (20.007) y los resultados de `fn_autenticar`, algo que el log no puede mostrar porque registra sentencias y no resultados.
- **Afirmó sin sustento** cuatro cosas: que `app_web` ya estaba autenticado (el log muestra la autenticación `scram-sha-256` en cada conexión), que la cuenta no se bloqueó (el log no muestra resultados), que el paso 3 era un sobre-permiso (las columnas son las que `rol_soporte` tiene por diseño) y que el acceso de `postgres` era remoto (era local; al anonimizar el host se perdió ese dato).

La secuencia es compatible con un ataque a una credencial, pero **el esquema de permisos resistió**: los tres intentos fallaron y no cambió ninguna membresía ni se creó ningún rol.

Decisión de contención. Es del equipo y no una acción automática de la IA; en el simulacro no se aplicó ninguna medida, describen qué se haría en un caso real. Se adoptan: preservar el log en copia de solo lectura, verificar `pg_auth_members`, `pg_roles` y privilegios, suspender el login con `NOLOGIN` (reversible, se prefiere a borrar el rol), forzar el cambio de contraseña de la cuenta atacada, bloquear el origen solo si no está autorizado, y mejoras a mediano plazo (bloqueo y alerta tras intentos fallidos en `fn_autenticar`, alerta por lecturas masivas de `usuario`, protección del archivo de log). Se descartan tres sugerencias de la IA: eliminar el usuario de prueba (es un artefacto del simulacro), tratar el paso 3 como sobre-permiso y dar por hecho un acceso remoto.

Límites conocidos.

- La anonimización dejó valores **autodescriptivos** (`CLAVE_OK_SINTETICA`, `usuario.sim.0001`) que

delatan un simulacro, y la IA los usó para favorecer la hipótesis de un ejercicio controlado; además, `IP_ANONIMIZADA` borró que el origen era la propia máquina.
- La spec (`design.md`, sección 7) preveía 3 logins fallidos y un login `soporte_prueba`; el simulacro ejecutado usó 5 intentos y `soporte_sim`. La diferencia es de ejecución, no de resultado.
- La evidencia viene de servidores y fechas distintas (el simulacro del 12/10 y la prueba de auditoría del 14/10); no hay una tabla de cadena de custodia.
- `fn_autenticar` no tiene bloqueo por intentos fallidos: es una mejora propuesta, no implementada.
- La versión actual del informe del incidente reemplaza a la anterior de Facundo; la reescribió Mariano con ayuda de Claude, sobre la respuesta de la segunda corrida.

4. Uso de IA en la unidad

- **OpenCode (Nemotron 3.5 Lightning):** auditoría de `roles.sql` (sección 2). Se le pasaron solo `roles.sql` y `design.md`, sin datos reales.
- **OpenCode (simulacro):** reconstrucción del incidente a partir del log anonimizado (sección 3.2), en dos corridas, la primera descartada. El modelo no está registrado en `informe_incidente.md`.
- **Claude:** revisión que originó el hallazgo H4b; apoyo para escribir los scripts de prueba; redacción de este informe y de `auditoria_roles_opencode.md`, y reescritura de `informe_incidente.md`. Todo se revisó y se comprobó contra las evidencias del repositorio. Los mensajes con que se retomó la auditoría de `roles.sql` tras dos interrupciones también los redactó Claude.
- **Dónde consta:** esta unidad no tiene DUIA propia; el registro está en `unidad-5/BITACORA.md`, en los registros crudos de OpenCode de `unidad-5/docs/` y en la bitácora unificada del repositorio (`docs/bitacora_ia.md`).
- Lo que se aceptó y lo que se descartó, con su motivo, está en la tabla de la sección 2.

Unidad 6 — Mantenimiento, respaldo y recuperación

Qué hicimos

Se amplió el plan de mantenimiento de PostgreSQL con respaldo diario programado, retención de 14 días, control de integridad y restauración de prueba semanal. El respaldo de `foodstore` usa el rol limitado `operador_respaldo`; genera un dump custom, un respaldo separado de objetos globales con `pg_dumpall --globals-only --no-role-passwords` y SHA-256 para ambos archivos. Las tareas `FoodStore-Respaldo-Diario`, `FoodStore-Mantenimiento-Semanal` y `FoodStore-Restauracion-Semanal` se registran con `programar_tareas.ps1` en el Programador de tareas de Windows (respaldo diario 02:00; mantenimiento y restauración los domingos 03:00 y 04:00). En la repetición del 07/11 se registraron, se dispararon a mano y se desregistraron (ver «Repetición y correcciones del 07/11»).

La restauración nunca toca `foodstore`: verifica primero los hashes y restaura únicamente en `foodstore_restore_test`. Luego ejecuta `ANALYZE`, `pg_amcheck --install-missing`, compara automáticamente los conteos de las cinco tablas principales entre origen y destino, y controla valores inválidos y claves foráneas huérfanas.

Por qué así

El dump custom permite verificarlo con `pg_restore --list` y restaurarlo de forma selectiva. Elegimos SHA-256 antes de la restauración para no destruir la copia descartable usando un archivo alterado. Se descartó ejecutar automáticamente el SQL de `pg_dumpall`: los roles son globales al servidor y restaurarlos desde una tarea programada podría afectar otras bases locales. Por eso se preserva y verifica su dump, mientras que la aplicación de roles y `verificacion_roles.sql` queda como una intervención manual aprobada por el equipo.

No se usa `VACUUM FULL`: el mantenimiento observa primero las estadísticas y el ensayo de `VACUUM (ANALYZE)` se realiza solo en la copia descartable.

Cómo lo probamos y resultados

- El rol `operador_respaldo` quedó sin `SUPERUSER`, `CREATEDB`, `CREATEROLE`, `REPLICATION` ni `BYPASSRLS`, y pudo leer `producto` para respaldar la base.
- El respaldo real generó el dump de 10.667.028 bytes, su SHA-256, el dump global sin hashes de contraseña y su SHA-256.
- La restauración válida verificó ambos hashes, ejecutó `pg_amcheck`, igualó los conteos de `categoria` (6), `usuario` (20.006), `producto` (50.016), `pedido` (200.006) y `detalle_pedido` (400.013), y obtuvo cero productos inválidos, detalles inválidos y referencias huérfanas.
- RTO observado: **7,11 s**. RPO observado: **0,05 h** (aprox. 2 min 52 s). Son valores de un laboratorio local, no una garantía productiva.
- En el simulacro de hash alterado, la restauración terminó con código 1 antes de modificar `foodstore_restore_test`; el OID de esa base no cambió.
- La retención eliminó un conjunto artificial completo de más de 14 días y conservó un conjunto vigente.
- La tarea de respaldo se ejecutó desde el Programador de tareas por disparo manual y devolvió `LastTaskResult = 0`. Esa evidencia no identifica la tarea ni la hora de ejecución; la repetición del 07/11 sí lo hace.

Evidencia

- Scripts y spec: `unidad-6/SPEC_U6_ampliacion.md`, `backup_foodstore.ps1`, `restore_test.ps1`, `crear_rol_respaldo.sql`, `validar_integridad.sql`, `programar_tareas.ps1`.
- Pruebas automatizadas: `unidad-6/tests/ampliacion.Tests.ps1` (8/8 aprobadas).
- Evidencias reales: `unidad-6/evidencias/obj7_*_20261107.txt`.
- Registro de IA, decisiones y fallos corregidos: `unidad-6/BITACORA.md`.

Límite conocido

La prueba se hizo en la misma máquina y con una base de aproximadamente 10,7 MB; por eso el RTO no se extrapola a producción. El respaldo global de roles se conserva y verifica, pero no se restaura automáticamente. Como control manual posterior a una restauración nueva de `foodstore_restore_test`, se aplicó `unidad-5/sql/roles.sql` y se ejecutó `verificacion_roles.sql`: T01 a T23 finalizaron en `OK`. La evidencia está en `unidad-6/evidencias/obj7_roles_aplicacion_20261107.txt` y `obj7_verificacion_roles_20261107.txt`.

Repetición y correcciones del 07/11 (Mariano Chirino, sobre el trabajo de Andrés Fabre)

Se revisó la rama `a/obj-07` leyendo `backup_foodstore.ps1`, `restore_test.ps1`, `crear_rol_respaldo.sql`, `programar_tareas.ps1` y `validar_integridad.sql`, y se corrigió y repitió en la rama `m/fix-obj-07`. La redacción de scripts y textos de esta revisión se hizo con asistencia de Claude y se verificó contra las salidas crudas.

Qué se corrigió. `restore_test.ps1` ahora cuenta y registra cada uno de los 6 controles de `validar_integridad.sql`, escribe `OK - 6 controles de integridad en cero.` y falla si no son exactamente 6. Antes, una salida vacía se tomaba como éxito.

Qué se repitió y qué mostró (máquina de Mariano, PostgreSQL 17, evidencia en `unidad-6/evidencias/obj7_*_mariano_*`):

- Respaldo y simulacro, 3 corridas: RTO 10,07 s, 7,75 s y 7,07 s; hashes verificados antes de crear la

base, `pg_amcheck` correcto, 6 controles en cero y conteos origen = destino en las cinco tablas.

- Roles sobre la copia restaurada con el `roles.sql` vigente (con las correcciones H1, H3 y H4b de la U5): `COMMIT`, 8 `WARNING` y 0 `ERROR`; `verificacion_roles.sql` T01 a T23: 23 de 23 `OK`.
- Programador de tareas (`obj7_programador_mariano_*`): las tareas figuraban sin ejecuciones previas; se dispararon con `Start-ScheduledTask` a las 19:56:08 (respaldo) y 19:56:12 (restauración). `LastRunTime` coincide con el disparo, `LastTaskResult = 0` y existen los logs `backup_20261107_195608` y `restore_20261107_195612`. Después se desregistraron, y no queda ninguna tarea `FoodStore-*`.
- Mantenimiento (`obj7_mantenimiento_*`): la primera corrida mostró la Fase 0 con `n_live_tup = 0` en todas las tablas de `foodstore`, porque la copia se creó por plantilla y no traía estadísticas; esa decisión «sin accion» no es concluyente. Se repitió tras un `ANALYZE` manual: con estadísticas reales (50.016 productos, 400.013 detalles, 200.006 pedidos, 20.006 usuarios), las seis tablas dan «sin accion» (un solo `n_dead_tup` en `producto`, contra un umbral de 10.053). En la base descartable, `n_dead_tup` de `producto` pasó de 0 a 50.015 tras el `UPDATE` y volvió a 0 tras el `VACUUM (ANALYZE)`.
- Pester (`obj7_pester_mariano_20261107.txt`): 8 de 8 con Pester 3.4.0. Esas pruebas solo buscan texto en los scripts; no prueban comportamiento. El comportamiento lo muestran las corridas anteriores.

Decisión de diseño sobre los roles globales (spec U6-A09). La restauración automática no aplica el dump de globales: crear roles es una acción global del servidor. Solo verifica su existencia y su hash. Los roles se aplican a mano sobre la copia con `roles.sql` y se verifican con `verificacion_roles.sql`.

Límites conocidos (ampliación)

- **Programador:** no se observó ninguna ejecución por horario; solo disparos manuales. Las tareas son `Interactive` y no usan `-StartWhenAvailable`: si la PC está apagada o sin sesión a la hora prevista, la ejecución se saltea. La tarea de mantenimiento no deja log propio y corre como `postgres`; su evidencia de contenido es la corrida directa. La evidencia original del programador no identificaba la tarea, y su `LastRunTime` (02:38:39) no coincide con el inicio de ningún log.
- **RPO:** el valor observado mide la antigüedad del dump al simular. En las repeticiones dio 0,00 h porque el simulacro corrió segundos después del respaldo. El RPO real depende de la frecuencia del respaldo programado (diaria).
- **Mantenimiento:** la parte que escribe (`UPDATE` y `VACUUM`) es un ensayo sobre la base descartable; sobre `foodstore` solo se observa. Se ejecutó un `ANALYZE` manual en la copia de Mariano.
- **Autenticación:** el `pg_hba.conf` de la máquina de Mariano usa `trust`; la evidencia no prueba la autenticación del rol de respaldo.
- **Respaldo y retención:** si `pg_dumpall` falla después de renombrar el dump, queda un conjunto incompleto que la retención ignora y no borra. El dump global requiere superusuario. Los conteos de origen se toman del `foodstore` vivo contra un dump anterior: un pedido nuevo en el medio daría un falso error. La verificación de hashes queda fuera del RTO. `ALTER DEFAULT PRIVILEGES` de `operador_respaldo` vale solo para el rol que corre el script.
- **Evidencia:** la evidencia de Andrés tiene rutas editadas (`<RAIZ_REPO>`) y su prueba de roles corrió contra un `roles.sql` anterior a las correcciones H1, H3 y H4b; por eso se repitió. Las evidencias de Mariano conservan las rutas locales sin editar. El log `obj7_repeticion2_mariano_restore_20261107_194601.txt` quedó interrumpido durante `pg_restore` (causa no registrada) y se conserva con su nota. Las salidas de `psql` en español traen `ï¿½` en las líneas «Ahora está conectado…» por un problema de codificación de la consola; no afecta a los datos y no se editó.

Unidad 7 — Comparación documental y relacional medida

Para complementar la migración original de siete documentos se creó un benchmark aislado de 5.000 pedidos vigentes. PostgreSQL siguió siendo la fuente de verdad y la carga fue de solo lectura; MongoDB usó la base separada `foodstore_nosql_benchmark` y la colección `pedidos_benchmark`, sin alterar la entrega didáctica original.

La muestra coincidió en ambos motores: 5.000 pedidos/documentos y un total de 68.818.099,92. También coincidieron las ventas de las cinco categorías y la verificación en PostgreSQL no encontró referencias a usuarios ni productos inexistentes. En la primera medición (una sola corrida), MongoDB creó un índice multikey sobre `lineas.producto_id`; la consulta por producto devolvió un documento y examinó una clave y un documento (`executionTimeMillis: 0`, por debajo de la resolución de 1 ms). PostgreSQL devolvió una fila en 141,2 ms, con 5.524 bloques compartidos en caché y 8.558 leídos. Esa medición tiene límites que declaramos: el `EXPLAIN` de PostgreSQL incluye construir la muestra y elegir el producto más frecuente (con un recorrido secuencial de 400.011 filas de `detalle_pedido`), mientras que en MongoDB ese producto se calculó antes y fuera de la medición; PostgreSQL leyó bloques de disco y MongoDB trabajó con la colección ya cargada; y fue una sola corrida.

Para cubrir esos límites, y porque la variante con la categoría embebida no se había medido, se repitió la medición el 07/11 sobre una base propia (`foodstore_u7_bench`, PostgreSQL 17.10, MongoDB 9.0.4). Como `carga_masiva.sql` usa `random()` sin semilla, los importes difieren de los de la primera medición (total de la muestra 68.802.975,31), y por eso los números de ambas no se mezclan. En esa base los dos motores volvieron a coincidir: 5.000 documentos, el mismo total, las mismas cinco ventas por categoría (también con `$lookup`) y los mismos conjuntos de usuarios (4.996) y de productos (10.001) según cantidad y suma de ids. Con cinco corridas tras una de calentamiento, la mediana de la consulta por producto fue 1 ms en MongoDB con el índice multikey y 58,6 ms en PostgreSQL. En ventas por categoría, la mediana fue 12 ms con la categoría embebida, 509 ms con `$lookup` y 79,4 ms con los JOINs de PostgreSQL. En esa muestra ningún producto aparece en más de un pedido, así que la consulta por producto es una búsqueda puntual.

No concluimos que un motor sea más rápido en general: son mediciones locales, en una sola máquina, con 5.000 pedidos, tiempos de reloj con resolución de 1 ms, planes de distinta forma y sin capturar el plan de PostgreSQL en la repetición. Lo que sí respaldan es que el índice multikey permite localizar el pedido embebido sin combinaciones, y que resolver la categoría con `$lookup` fue más lento que tenerla embebida y que los JOINs de PostgreSQL; embeberla elimina ese costo a cambio de duplicar el nombre de la categoría en cada línea (desnormalización de lectura, no una nueva fuente de verdad). Conservamos PostgreSQL para escrituras, stock e integridad, y MongoDB solo como copia opcional para lecturas de pedidos completos.

Evidencia: `unidad-7/evidencias/comparacion_medida.md` y `obj10_*_20261107.txt`. Scripts: `modelo/exportar_pedidos_benchmark.sql`, `carga_benchmark_mongodb.js`, `consultas_benchmark_postgresql.sql`, `medicion_repetida_mongodb.js`, `medicion_repetida_postgresql.sql` y las verificaciones asociadas.

Unidad 8: la base de la tablet y su sincronización simulada

Objetivos que cubre esta sección: **11** (esquema y sincronización especificados en Kiro y generados con OpenCode) y **12** (sincronización simulada con conflictos probados y resueltos). El code review (objetivo 13) está en `codereview.md` y la capa de datos integrada (objetivo 14) en el README raíz.

1. La base de la tablet y el semáforo (objetivo 12)

Qué hicimos. Una base SQLite para la tablet del food truck, que trabaja sin conexión: `producto_cache`

(copia del catálogo), `pedido_borrador` e `item_borrador` (lo que anota la tablet), `cola_sync` (solicitudes a sincronizar) y `snapshot_servidor` (tabla de paso para importar la instantánea). Las cinco tablas son `STRICT`, con los importes en centavos. La instantánea sale de PostgreSQL por la vista `v_snapshot_movil` (sin datos de usuarios) y se exporta a CSV. El semáforo (`semaforo.sql`) está escrito en SQL puro: evalúa solo las solicitudes `PENDIENTE` y las marca `CONFLICTO`, `RECHAZADO` o `SIMULADO_APROBADO`.

Por qué así, y qué descartamos.

- **`UPSERT` y no `INSERT OR REPLACE`** para recargar el catálogo: `REPLACE` borra la fila y se perdería la baja lógica (lápida) y la referencia desde los borradores.
- **Índice parcial `ux_cola_una_pendiente` y no un `UNIQUE` simple** sobre `cola_sync.id_local`: el `UNIQUE` simple impediría reenviar una solicitud después de un `CONFLICTO`.
- **`LEFT JOIN` y no `INNER JOIN`** en el semáforo: con `INNER JOIN` un producto que no existe en el servidor pasaría como verde.
- **Semáforo en SQL y no en Python:** el `simulador_sync.py` que reconstruyó una IA comparaba una tupla con un texto y nunca habría evaluado nada.
- **El rojo gana sobre el amarillo**, y un producto ausente de la instantánea es rojo. `SIMULADO_APROBADO` **no es una venta**: el móvil propone, el servidor dispone.

Cómo lo probamos. Casos C1 a C8 y casos borde F2 sobre SQLite 3.54.1 y una copia de PostgreSQL 17.10 con datos inventados, con la salida de cada comando guardada en `unidad-8/evidencias/`:

- C1: el catálogo sin servidor muestra **13** de los 16 productos (se ocultan el 3, el 15 y el 16).
- C2: sube el precio en el servidor, la solicitud queda en `CONFLICTO` y, tras aceptar el precio nuevo, una solicitud nueva queda `SIMULADO_APROBADO`.
- C3 (stock insuficiente) y C4 (producto dado de baja): `RECHAZADO`. C5 (control): `SIMULADO_APROBADO`.
- C6: reenviar el mismo UUID falla por clave primaria y la fila original no cambia.
- C7: tres inserciones inválidas (cantidad 0, producto inexistente, tipo inválido en columna `STRICT`) fallan y no dejan filas.
- **C8 (el caso que muestra el límite del móvil):** dos borradores piden 4 unidades cada uno de un producto con stock 6. Cada uno pasa solo y juntos piden 8. El semáforo evalúa uno por vez y deja a los dos en `SIMULADO_APROBADO`: solo el servidor puede resolver la competencia (con `sp_crear_pedido` y `SELECT … FOR UPDATE`).
- Prueba de que PostgreSQL no cambió: `verificar_no_cambio.sql` antes y después da la misma huella de stock (`89f8351d0162e23ef1473f32b05ce221`; 6 pedidos, 13 detalles, stock 454).

Límite conocido. El semáforo valida solo la longitud del UUID, no su formato. `casos.sql` no se puede repetir sobre la misma base (usa UUID fijos), por eso la spec pide una base nueva.

2. Esquema y sincronización generados con OpenCode desde la spec (objetivo 11)

Qué hicimos. El enunciado pide que la spec se escriba en Kiro antes y que OpenCode genere los scripts. Los scripts originales (`unidad-8/movil/`) no se hicieron así: los escribimos con Claude y NotebookLM, y la primera versión de la spec (03/11) fue posterior al código. Lo declaramos en la DUIA y lo corregimos regenerando:

1. Ampliamos la spec en Kiro y la commiteamos **antes** de generar (`be854e4`, 06/11 15:38).
2. **Corrida 1** de OpenCode (agente Nemotron 3.5 Lightning Free, 06/11, 16:10): dándole solo `AGENTS.md` y la spec. No produjo una base funcional: el esquema ni siquiera se creó (`STRICT` mal ubicado y un `UNIQUE … WHERE` dentro del `CREATE TABLE`) y fallaron en cascada la carga, el semáforo y los casos. Los originales se conservan sin editar.
3. La versión que funciona (`movil_opencode/`) la corregimos con Claude a partir de esos errores, leyendo y ejecutando cada cambio, y lo declaramos. Pasa C1 a C8 y F2.

4. Al compararla con `movil/` aparecieron **dos lagunas de la spec** (no del modelo): faltaban un valor por defecto en `pedido_borrador.creado_en` y la clave foránea de `cola_sync.id_local`. Primero se agregaron a la spec (REQ 6.12 y 6.13) y después al esquema, cada paso con su commit.
5. **Corrida 2** (07/11, 09:39 a 09:50): mismo modelo, en una carpeta aislada que no contenía `movil/`, sin editar el resultado (`movil_opencode_v2/`). Tampoco funcionó.

Por qué dos corridas. La corrida 1 no permite saber si OpenCode miró los originales (el prompt lo prohibía, pero no se puede verificar). La corrida 2, en una carpeta sin `movil/`, mide lo que genera el modelo solo desde la spec.

Resultado de la corrida 2 (evidencias 28 y 29, `docs/comparacion_generado_vs_original.md`, sección 8): nueve defectos. Los más graves: `cargar_catalogo.sql` repite el error de sintaxis de la corrida 1; **ninguna de las cinco tablas es `STRICT`** (strict = 0); falta el `CHECK` que exige `motivo` en `CONFLICTO` y `RECHAZADO`; seis scripts no activan `PRAGMA foreign_keys`; hay errores de tipeo (`criada_en`) y sintaxis de otro motor (`BEGIN TRY/CATCH`). El agente afirmó en su respuesta «5 tablas STRICT» y «sintaxis revisada», y ambas afirmaciones son falsas (reconoció además no haber ejecutado nada). Lo que sí sirvió: `semaforo.sql`, los valores por defecto y la clave foránea (que la spec de esta corrida ya pedía).

Qué aprendimos.

- Una afirmación de la IA («tablas STRICT», «sintaxis revisada») se ejecuta y se comprueba; no se acepta por lectura.
- Cada `sqlite3 base ".read …"` abre una conexión nueva con las claves foráneas desactivadas, por eso `PRAGMA foreign_keys = ON;` va en **cada** script. Lo comprobamos por ejecución (evidencia 29): sin el `PRAGMA` entra un ítem de un producto inexistente y con el `PRAGMA` falla.
- Comparar con los originales mostró lagunas de la spec que la lectura no había mostrado.

Evidencia. `unidad-8/docs/comparacion_generado_vs_original.md`, `docs/registro_opencode_corrida1.txt` y `docs/registro_opencode_corrida2.txt` (prompt y respuesta cruda), `evidencias/22` a `29`, `movil_opencode/original_opencode/` y `movil_opencode_v2/` (sin editar), y la DUIA y la bitácora de Mariano.

Límites conocidos, dichos con claridad.

- Ninguna de las dos corridas produjo una base funcional sin correcciones. La versión final (`movil_opencode/`) combina OpenCode con correcciones hechas con Claude, y en su mayor parte son reescrituras.
- No se puede verificar si OpenCode leyó `movil/` durante la corrida 1.
- La primera versión de la spec fue posterior al código. La de la regeneración se commiteó antes de generar, pero no se escribió «desde cero»: los Requisitos 5 a 10 los redactó Claude como borrador y se pegaron casi sin cambios en Kiro. No consta con qué herramienta se redactó el Requisito 11.
- Dos diferencias declaradas y sin resolver, sin efecto en C1 a C8 ni en F2: la vista `v_catalogo_disponible` no muestra `capturado_en`, y la spec no pide que el script de casos informe `OK` o `FALLA` según el resultado real.
- No contamos con una respuesta registrada del profesor sobre cómo tratar el objetivo 11.

Code review con IA de los scripts (objetivo 13)

Autor: Mariano Chirino (la tarea estaba asignada a Facundo Quiroga y la hizo Mariano a partir del 08/11/2026; ver «Declaraciones»). Rama: `review-final`. Etiqueta de scripts congelados: `pre-review` (`351ddb9`, sobre `main`). Tabla completa: `docs/auditoria_codigo/hallazgos.md`. Respuestas crudas: `docs/auditoria_codigo/tanda_*.md`. Evidencia: `docs/auditoria_codigo/evidencias/`.

1. Qué se hizo

Se pasó el código del repositorio por OpenCode (modo Plan) en 11 tandas, con el mismo prompt base (`prompt_base.md`) y guardando la respuesta cruda de cada una. Cada hallazgo devuelto se cargó como una fila de `hallazgos.md` con archivo y línea, resumen, severidad informada por la IA, autor, decisión, motivo y prueba.

Tanda	Contenido	Filas
01	`heredado/base_heredada` (la primera corrida falló por un marcador sin reemplazar y se repitió)	48
02	unidad-4	16
03	unidad-4/migracion	14
04	unidad-5, roles	16
05	unidad-5, pruebas	27
06	unidad-6, scripts PowerShell	17
07	unidad-6, SQL	11
08	unidad-7, consultas	11
09	unidad-7, benchmark	11
10	unidad-8, móvil y PostgreSQL	9
11	unidad-8, móvil generado por OpenCode (corrida 2)	16
	Total	**196**

Las 27 filas de la tanda 05 se armaron después, a partir de la prosa de la respuesta, porque esa corrida no devolvió ids ni severidad.

Severidad informada por la IA (no validada por nosotros): 2 críticas, 33 altas, 73 medias, 61 bajas y 27 sin informar.

2. Resultado de la revisión

Se decidió sobre **42 de las 196 filas**: las 35 de severidad alta o crítica y 7 de severidad menor que se tocaron al reproducir o al leer lo vecino (`T06-H-17` y `T11-H-11` a `T11-H-16`). Las otras **154 quedan «No evaluado»**: no se reprodujeron ni se verificaron contra el código por el plazo de entrega, y lo que devolvió OpenCode queda en la columna Resumen sin validar.

Decisión	Filas	De ellas, altas o críticas
Confirmado	31	25
Confirmado en parte	6	6
Parcial	2	2
Descartado	2	1
Descartado en lo central	1	1
No evaluado	154	0
Total	**196**	**35**

«Confirmado» incluye filas que dicen «parcial», «por lectura» o «con simulación» en su Decisión: el conteo no separa cómo se confirmó. La separación correcta es esta:

- **Con ejecución (14):** `T06-H-01`, `T06-H-02` (con un archivo de prueba, no con el script real), `T06-H-17` (comprobación de la versión de Pester), `T01-H-34` (el comando solo, no el archivo) y `T11-H-01` a `T11-H-06`, `T11-H-08` a `T11-H-11`. `T11-H-08` y `T11-H-09` se probaron con el paso 6 del README simulado, no con el real de PostgreSQL; `T11-H-06` es parcial (no se probó el valor de `changes()`).

- **Por lectura de código (28):** el resto de las 42. Una confirmación por lectura dice que el código hace lo que el hallazgo afirma, no que el efecto se haya observado.

3. Lo que la revisión corrigió de OpenCode

Tres hallazgos de severidad alta no se sostienen o cambian de sentido:

- `T06-H-02` **(descartado).** El hallazgo decía que la tarea de mantenimiento sigue tras errores SQL y reporta éxito. `maintenance.sql:33` tiene `\set ON_ERROR_STOP on` antes de cualquier sentencia y la tarea termina con `exit $LASTEXITCODE`. Con un archivo de prueba, `psql -f` se detiene en el primer error y sale con código 3. No se corrió el `maintenance.sql` real con un error forzado.
- `T07-H-01` **(descartado en lo central).** El rol de respaldo se crea sin contraseña a propósito (lo dice el script) y la contraseña se fijó aparte; la bitácora de la U6 registra que `pg_dump` corrió con ese rol. Queda que el paso manual no está en el script.
- `T06-H-17` **(descartado, severidad baja).** `Should Be` es la sintaxis válida en el Pester 3.4.0 del entorno y las 8 pruebas pasan; la deprecación es de Pester 5, que no está instalado.

Y otros cambian de formulación, de alcance o de gravedad:

- `T01-H-09`**:** la función sí toma el precio de `producto` cuando falta; lo que acepta es un precio manual.
- `T01-H-31`**:** ninguna de las tres consultas filtra por estado; el filtrado de eliminados es parcial (la A no une `pedido`, la C no filtra `usuario`).
- `T01-H-41`**:** es cierto que la carga es una sola transacción; eso da atomicidad y no se probó que sea un defecto. Severidad a revisar.
- `T07-H-02`**:** un `pg_dump` completo necesita leer todas las tablas; lo que queda es proteger el archivo de respaldo.
- `T08-H-08` **y** `C-05`**:** el script no usa `$env:PGPASSWORD` (solo la nombra en un comentario); lo que falta son `-w` y `-X` en las llamadas a `psql`.
- `T09-H-02` **y** `T09-H-03`**:** el hecho es cierto; como defecto es discutible (un pedido histórico conserva su producto y su cliente aunque hoy estén dados de baja).
- `T11-H-16`**:** el resumen de OpenCode es incorrecto para v2: `creada_en` y `creado_en` usan el mismo formato; la inconsistencia es con `evaluada_en`.
- `T01-H-33` **y** `T01-H-35`**:** son guías escritas para dos sesiones; corridas en una sola no prueban aislamiento ni bloqueo, y el archivo no lo advierte.

Lo que sí confirma la revisión sobre el código generado por OpenCode (tanda 11, unidad-8/movil_opencode_v2)

Las dos corridas de OpenCode para el móvil no entregaron nada ejecutable sin correcciones (ver T1). En la corrida 2 se reprodujo, entre otros: que las tablas no son `STRICT` aunque el encabezado y el README lo afirman (`T11-H-01`); que un script usa sintaxis de T-SQL inexistente en SQLite (`T11-H-03`); que un script de pruebas no ejecuta el semáforo y deja los seis casos F2 en `PENDIENTE` (`T11-H-07`); y que `PRAGMA foreign_keys` falta en casi todos los scripts (`T11-H-11`).

4. Hallazgos propios (no de OpenCode)

Las filas `C-01` a `C-09` registran lo que Mariano y Claude encontraron por su cuenta y no figura en la salida de OpenCode. `C-05` está decidida (confirmado en parte); el resto sigue pendiente de cargar o de decisión. Entre ellos: la restauración pierde dueños y permisos (`C-02`), el respaldo no verificaba su SHA-256 antes del 07/11 (`C-03`), y el «sin acción» original del mantenimiento pudo deberse a estadísticas en cero tras `createdb -T` (`C-06`).

5. Límites

1. **154 de 196 hallazgos no se evaluaron.** El informe no afirma nada sobre ellos más que la severidad

que informó la IA.
2. **Las altas y críticas se decidieron en su mayoría por lectura.** Solo 14 filas tienen ejecución propia.
3. **Tres hallazgos dependen de lo que pide el enunciado y no se contrastaron con él:** `T01-H-01`, `T01-H-31` y `T09-H-01`.
4. **Las pruebas con paso 6 simulado** (`T11-H-08`, `T11-H-09`) no equivalen a correr el paso 6 real en PostgreSQL.
5. **Una sola IA revisora** (OpenCode con un modelo gratuito); no hubo una segunda revisión independiente. En las cabeceras de las tandas el modelo figura como «Croutou Alpha Free» (01, 07 y 08) y «Crouton Alpha Free» (09, 10 y 11); no se pudo verificar cuál es el nombre exacto.
6. **No se corrigió código a partir de la revisión** en esta etapa: ningún script de `main` fue modificado por este trabajo (`pre-review` sigue siendo la referencia).
7. **Las decisiones se cargaron por lote** y no con un commit por hallazgo; la evidencia sí está separada por grupo.

6. Declaraciones

- La tarea T9 se reasignó de Facundo a Mariano el 08/11 y se ejecutó entre el 08 y el 09/11/2026.
- Las respuestas de OpenCode se guardaron crudas. El 09/11 se completaron los campos Modelo, Fecha, Estado y Total en las cabeceras de las tandas 01 a 06 (commit `e815760`); las respuestas no se editaron.
- La preparación de las reproducciones, el armado de las filas de `hallazgos.md` y los scripts de carga se hicieron con Claude (Anthropic); Mariano ejecutó cada comando y revisó las salidas.
- La evidencia `T06-H-01_pester_sin_llamada.txt` es un `Start-Transcript` y trae usuario, equipo y ruta locales en su cabecera. `T06-H-17_version_pester.txt` quedó en UTF-16 por estar ya commiteada; el resto de la evidencia nueva está en UTF-8.
- Los acentos rotos de algunas salidas vienen de la consola de PowerShell 5.1 y no se editaron.
- `T11-H-11_f2_huerfanas.txt` se conserva pero no se usa como prueba de `T11-H-11`: `producto_cache` estaba vacío en esa corrida.

4. Uso de IA

Herramientas

- **Kiro**: especificaciones (requisitos, diseño, tareas) antes de generar código.
- **OpenCode** (modelo Nemotron 3.5 Lightning Free en las corridas de la U8 y de la auditoría de `roles.sql`): generación de la base SQLite y auditoría de permisos.
- **Claude** (chat): borradores de requisitos, corrección de los scripts generados, revisión de hallazgos y redacción de documentos, siempre revisados contra las evidencias.
- **Gemini y NotebookLM**: usados en versiones anteriores de la U4 y la U8, declarados en las DUIA de cada unidad.

Reglas que se respetaron

Nada generado se ejecutó sin leerlo línea por línea y probarlo en una copia o en una transacción reversible. No se enviaron datos reales a una IA. Cada corrida dejó registro de modelo, fecha, prompt, respuesta y decisión.

Qué pasó con cada corrida (objetivos 6 y 11)

Objetivo 11 (SQLite). La spec de Kiro se completó **antes** de generar (commit `be854e4`). Se hicieron dos

corridas de OpenCode y **ninguna produjo una base funcional sin correcciones**:

- Corrida 1 (06/11): no creó el esquema y falló en cascada. Los originales están en `unidad-8/movil_opencode/original_opencode/`.
- Corrida 2 (07/11, carpeta aislada sin acceso a `movil/`): error de sintaxis en la carga del catálogo, **ninguna tabla era `STRICT`** aunque el README lo afirmaba, columna mal escrita, sintaxis de SQL Server en SQLite, falta de un `CHECK` y ausencia del archivo de casos. Dos afirmaciones de su respuesta («5 tablas STRICT», «sintaxis revisada») eran falsas: no había ejecutado nada. Queda guardada sin editar en `unidad-8/movil_opencode_v2/`.

La versión final es `movil_opencode/`, corregida con Claude y declarada como tal. Al comparar con los scripts originales aparecieron dos lagunas de la spec (REQ 6.12 y 6.13), que se cerraron con su propio commit.

Objetivo 6 (auditoría de `roles.sql`). La sesión original de OpenCode no se conserva; solo hay un resumen de 10 hallazgos en la bitácora. Se **repitió** la auditoría el 07/11 sobre la versión actual (modo Plan, carpeta aislada, respuesta cruda guardada en 3 partes por 2 interrupciones). Devolvió 5 hallazgos, todos reproducidos en una copia antes de decidir:

Hallazgo	Decisión	Por qué
H1 `CREATE` en `public`	Aceptado y corregido	Con base limpia no se reproducía; con un `GRANT CREATE` previo, el permiso sobrevivía a `roles.sql`
H2 contraseña en re-ejecución	Descartado	El hash SCRAM es idéntico antes y después; la corrección propuesta pisaría claves reales
H3 membresías entre grupos	Aceptado y corregido	El residuo de una ejecución previa dejaba a `rol_soporte` ejecutando `sp_crear_pedido`
H4 `EXECUTE` indirecto	Descartado	0 filas para `PUBLIC`; el razonamiento de la IA era incorrecto
H5 default privileges	Descartado	La afirmación sobre PostgreSQL 15 es falsa

Además, **H4b** (cuatro rutinas viejas con `EXECUTE` para `PUBLIC`) **no salió de OpenCode sino de una revisión con Claude**; se declara así, se reprodujo y se corrigió. Con las tres correcciones, las pruebas T01 a T23 dan 23/23.

Lecciones que cambiaron cómo trabajamos

- Una prueba que pasa no prueba lo que se cree: la de H2 no discriminaba porque `pg_hba.conf` usa `trust`. Antes de citar una evidencia, se verifica qué mide.
- Los dos hallazgos que OpenCode acertó (H1 y H3) eran de base sucia: la copia limpia no los mostraba. No se aprueba ni se descarta un hallazgo por lectura.
- Si una IA afirma algo, se ejecuta y se comprueba.
- `PRAGMA foreign_keys = ON;` en cada script de SQLite.

Redacción del informe

Los textos de este informe y el documento `auditoria_roles_opencode.md` se redactaron con asistencia de Claude y se revisaron contra las evidencias del repositorio. Los textos de `u4.md` se redactaron con Claude (declarado en la DUIA de Facundo, 08/11/2026) y Mariano los corrigió con Claude ese día. `u6.md` lo amplió Mariano con Claude el 07/11/2026 sobre el texto de Andrés. De `u7.md` no consta con qué herramienta se redactó.

5. Limitaciones y trabajo futuro

Del trabajo con IA

- Ninguna corrida de OpenCode entregó algo ejecutable sin correcciones; la versión final de la U8 depende de correcciones hechas con Claude.
- La auditoría de `roles.sql` es una repetición: la respuesta cruda de la original no existe. De los 10 hallazgos originales solo H1, H2, H4, H6 y H7 están identificados; del resto «no consta».
- No consta con qué herramienta se redactó el Requisito 11 de la spec.
- El registro de la auditoría llegó en tres partes; los mensajes de las interrupciones los redactó Claude y no se compararon carácter por carácter. Las horas de las partes 2 y 3 «no constan».

De los permisos (U5)

- `pg_hba.conf` de la copia usa `trust`: T01 a T23 verifican permisos, no autenticación.
- El marcador de contraseña `'cambiar_en_produccion'` es público.
- El `REVOKE` de H3 solo quita membresías concedidas por `postgres`; no se probaron `GRANT OPTION` de otros otorgantes.
- La prueba de H4b no ejercita una ruta donde `app_web` dispare las funciones con sus propios permisos.
- Los roles no cubren los objetos de la U4 (`control_lote`, `responsable_deposito`, `mv_ventas_categoria_dia`): la ampliación de los roles a esos objetos (T8) no se hizo.
- `fn_autenticar` no bloquea por intentos fallidos; `log_statement='all'` deja claves en el log.

De la base SQLite (U8)

- `capturado_en` en `v_catalogo_disponible` y el criterio `OK/FALLA` del Requisito 11 quedan como diferencias sin resolver entre la versión generada y la original; no cambian los casos C1 a C8 ni F2.
- No consta si OpenCode leyó `movil/` en la corrida 1, y no se ejercitó C2 en la corrida 2.
- El semáforo evalúa cada borrador por separado: dos borradores que compiten por el mismo stock pueden quedar ambos aprobados en simulación (C8). Solo el servidor resuelve.
- El semáforo valida la longitud del UUID, no su formato.

De los otros objetivos

- U4 (objetivos 1 a 3): la spec de la migración se actualizó después del código; no hubo pasada previa con `ROLLBACK`; la migración no preserva `{L,D} → R`; el paso «contraer» queda pendiente y mientras tanto hay dos fuentes de verdad; el `down.sql` repetido no tiene captura propia; la escala es de 3 filas. Las capturas del ciclo `up`/`down` son de PostgreSQL 18 y las de los caminos de falla, de PostgreSQL 17. Al 08/11/2026 no consta si el primer borrador de la Parte 1 lo hizo Gemini ni quién sacó las capturas del ciclo.
- U6 (objetivo 7): las tareas se dispararon a mano con `Start-ScheduledTask`; ninguna se ejecutó por horario. El mantenimiento no deja log propio y corre como `postgres`; `pg_hba` está en `trust`; el RPO de la prueba no es real; las pruebas de Pester solo buscan texto en los scripts. La primera corrida de mantenimiento dio «sin acción» con estadísticas en cero (copia hecha con `createdb -T`); se repitió tras `ANALYZE`, y falta revisar si la entrega original tuvo la misma causa.
- U7 (objetivo 10): los límites están en la fila 10 del `README.md` y en `unidad-7/evidencias/comparacion_medida.md`.
- Code review (objetivo 13): OpenCode en modo Plan, 11 tandas, 196 hallazgos. Se decidió sobre 42 (las 35 altas y críticas y 7 más) y 154 quedan «No evaluado». Solo 14 decisiones tienen ejecución propia; el resto es por lectura. Ningún script se modificó. Ver `docs/informe/codereview.md`.

De los datos de ejemplo y las evidencias

- La base heredada del repositorio usa usuarios inventados. Las exportaciones de la U7 (`unidad-7/modelo/pedidos_export.json` y la salida guardada en `unidad-7/evidencias/mongo_migracion_y_consultas.txt`) se regeneraron el 07/11/2026 sin el nombre real de un integrante (según `unidad-7/CONTEXTO_IA.md`). El `git grep` del 08/11/2026 no encuentra nombres de integrantes en esos dos archivos. Sí quedaba un resto en `unidad-7/AGENTS.md` (la fila del pedido 5 nombraba a un integrante como cliente), corregido el 08/11/2026.
- Las evidencias de las U5, U6 y U7 conservan rutas locales (`C:\Users\...`) y nombres de equipo (`MarianoChirino`, `DESKTOP-ANDRES`) de los integrantes. No se editaron, para no alterar lo que prueban.

Del proceso

- Los PR de `m/obj-11`, `m/informe-u8` y `m/obj-06` se mergearon **sin revisión cruzada**, porque la revisión asignada no respondió. Queda pendiente una lectura posterior.
- Las evidencias de H3 y H4b se generaron en UTF-16 y se reencodearon a UTF-8 sin cambiar el contenido (commit `6346e09`).

Con más tiempo: probar autenticación real (sin `trust`), ampliar los roles a los objetos de la U4, repetir las mediciones de la U7 con volumen, y automatizar la verificación de integridad también para SQLite.

6. Bases de datos, cómo reproducir y notas

Bases de datos usadas

Unidad	Base	Motor
U4	`copia_trabajo` (Parte 2) y `copia_fnbc` (Parte 1); `copia_u4` (migración y caminos de falla)	PostgreSQL 18 (`copia_trabajo`, `copia_fnbc` y las capturas `migracion_up.png` y `migracion_down.png`); PostgreSQL 17 (`copia_u4`, capturas `migracion_falla_*`)
U5	`copia_u5`	PostgreSQL 17.10
U6	`foodstore`, `foodstore_restore_test`	PostgreSQL 17 (local, Mariano y Andrés)
U7	`foodstore_nosql`	MongoDB 9.0.4 (`mongosh` 2.14.0)
U8	`foodstore_u8`	PostgreSQL 17.10 (Mariano; Andrés declara 17.11 en sus corridas del 05/11) + SQLite 3.54.1

Cómo reproducir

1. Cargar la base heredada en este orden: `schema` → `objects` → `data` → `views` → `materializadas.sql`. Conteos esperados: 6 / 16 / 6. **No** correr `carga_masiva.sql` salvo en la medición de la U7, y en una base aparte.
2. `unidad-5/sql/roles.sql` depende de `mv_facturacion_categoria_mes`: cargar `materializadas.sql` antes.
3. En SQLite, `PRAGMA foreign_keys = ON;` va en **cada** script (cada `sqlite3` es un proceso nuevo y arranca en OFF).
4. Ejecutar los comandos desde la raíz del repositorio y los scripts desde archivo (`-f`), no pegados en la consola.
5. En Windows, `psql` no está en el PATH: `& "C:\Program Files\PostgreSQL\17\bin\psql.exe"`.
6. Toda prueba corre en una copia o en una transacción reversible.

Nota sobre este PDF

Se armó el 09/11/2026 con asistencia de Claude (Anthropic) a partir de los textos de `docs/informe/` (`00_introduccion_uso_ia_limitaciones.md`, `u4.md` a `u8.md`, `codereview.md`) y del `README.md`, copiados sin reescribir. Lo único que se agregó o cambió: la portada y esta nota, el link y la etiqueta, la nota sobre el PDF de la U7 y la fila 14 de la tabla de objetivos. Los niveles de los encabezados se ajustaron para unificar el documento.